Student Project Proposal

1 Project and course details

Project title: Pulse: Timetable-Aware Commute Pooling for University Campuses (working title)

Institution and department: American International University-Bangladesh (AIUB), Department of Computer Science, Faculty of Science and Technology

Course code and title: CSC4161, Advanced Programming in Web Technology

Semester and section: Fall 2026-2027, Section B

Submitted to: [Instructor name and designation]

Team name or number: [Team name or number]

Submission date: [DD Month YYYY]

2 Team members

Student name	Student ID	Planned responsibility
Ashfak Hossain Evan	22-49947-3	Technical lead: architecture, event backbone and job queues, matching and optimisation algorithms, simulator, forecasting and request-understanding models, onboarding agent, GitHub pipeline
[Name]	[ID]	Identity, administration and money: sign-in, roles, profiles, preferences, admin panel, shuttle operator dashboard, wallet and ledger screens, safety centre
[Name]	[ID]	Routine, trips and quality: catalog and grade-report import, week view, go now, plan and Autopilot screens, crew and trip screens, notifications, end-to-end and accessibility testing

3 Project summary

Class timetables decide when students and faculty can arrive and leave, so the same people travel at similar times every week. At AIUB, 83% of class releases on regular teaching days fall at just ten clock times, yet each person still travels and pays alone. Pulse turns the official timetable into shared trips. Users import their routine from the semester grade report, then go now, plan a trip, or let Autopilot suggest their usual trips, typing requests in English or Banglish if they like. A forecasting model predicts who is likely to travel, optimisation algorithms form fair groups for hired vehicles, own cars and bikes, buses and simulated shuttles, and a game-theory method splits each fare. A campus simulator, an event-streaming backbone and replay testing make the system measurable. Campus verification, reliability scores, live trip safety and privacy-preserving design build trust. Every feature is free.

4 The problem this project solves

People on a campus leave at the same moments and in the same directions, yet each of them finds, waits for and pays for a vehicle alone. Students, faculty and staff commute across Dhaka every teaching day by CNG, rickshaw, ride-hailing car or crowded bus. Sharing happens only by chance: a classmate going the same way, or a message in a Messenger group.

The AIUB offered-course report for Fall 2026-2027 [1] (1,991 active sections, 71,921 enrolled seats) shows how predictable this demand is. A person can attend only one class at a time, so Figure 1 counts people walking out of class at each minute of a Sunday.

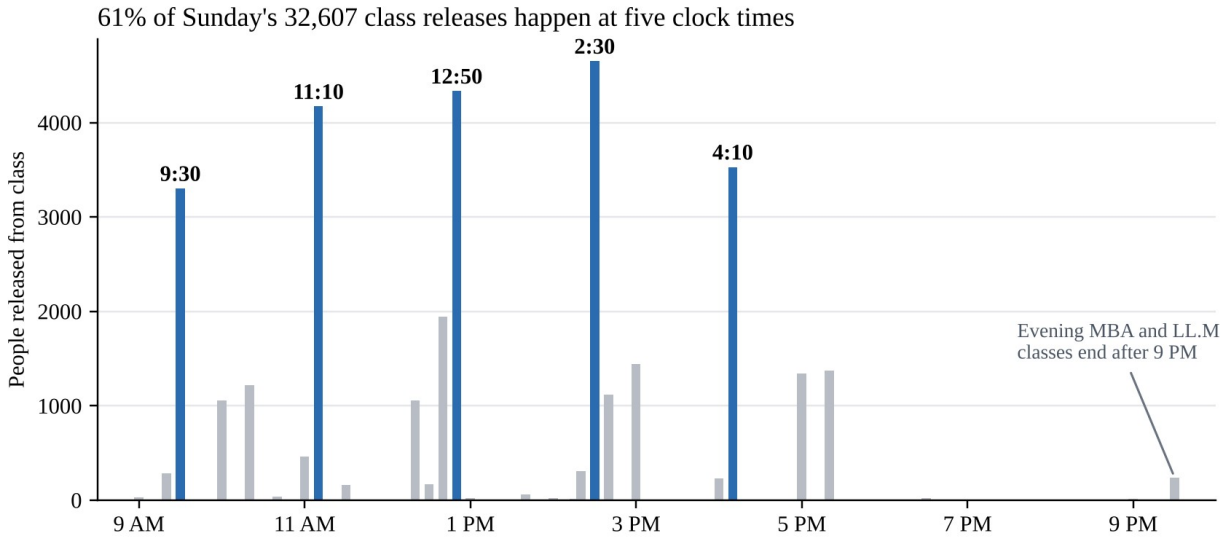


Figure 1. People released from class by time of day, Sunday, AIUB Fall 2026-2027. Cancelled and empty sections are excluded; Monday to Wednesday follow the same pattern.

Three further facts from the same report shape the design:

- **The week repeats.** 75% of sections meet on paired days (Sunday and Tuesday, or Monday and Wednesday), so most travel patterns repeat twice a week.
- **Mornings are a pulse too.** About 5,900 people start class at 8:00 AM every teaching day.
- **Late trips are common.** About 250 people leave evening classes after 9 PM every weeknight, when travelling alone feels least safe.

Existing sharing tools also fail in known ways. Carpool app users report last-minute cancellations and ride alerts that do not arrive [2]. Door-to-door pickups add detours that make matches fail, and riders trust people from their own institution far more than strangers [3]. Current apps match one trip at a time, as if requests were random, so they cannot use a timetable to know who is likely to travel, cannot react when a class is cancelled, and cannot settle many small shared fares between the same people.

One limit of this evidence: a class ending is not always a person leaving campus, because many students have a later class or stay on campus. The survey and pilot in Section 7 measure actual travel behaviour.

5 How this project helps

Pulse treats the timetable as a forecast of when each person is likely to travel, not as a fixed schedule. It knows the latest time someone must arrive and the earliest they are likely to leave, learns their real habits, and lets them travel at any time. The main journey for a student:

1. **Join.** Sign in with a code sent to the campus email address, so only verified members of the university can join.
2. **Import the routine.** Upload the semester grade report from the portal. It is read inside the browser, and only the current semester's class IDs reach the server, never grades. Faculty pick the sections they teach; staff enter office hours.
3. **Set preferences.** Drop a home pin, stored only as a coarse area, and choose allowed vehicles (CNG, rickshaw, ride-hailing car, own car or bike, bus), gender preference and maximum wait.

4. **Ride, in one of three ways.** Go now: request at any moment; the app shows how many people heading the same way are likely to leave soon and whether waiting a few minutes is worth it, and going alone is always offered at once. Plan for later: pick a time and place in advance. Autopilot: the app suggests each day's usual trips from the routine and real travel habits, and one tap confirms. A request can also be typed in English or Banglish, such as "ML class er por Mirpur 10 jabo, shudhu meye rider".
5. **Meet the crew.** The app shows who is going and why they were matched, the vehicle, a walkable meeting point and the booker. For a ride-hailing car, the booker taps one button to open Uber or Pathao with the trip filled in. Students and faculty can share a crew.
6. **Ride safely.** The crew chats in-app and shares live location during the trip. If the vehicle leaves the expected route or nobody confirms safe arrival in time, the app checks in and then alerts a trusted contact.
7. **Settle.** The booker enters the fare. It is split fairly with a game-theory method and recorded in a double-entry ledger that settles weekly with the fewest possible transfers.

On busy routes at busy times, the system also plans shuttle runs: seats on an operator's vehicle that riders book in advance. Operators are simulated through an operator dashboard in this version. When a class is cancelled, moved or added, one report updates the timetable and every affected plan is re-made, with members notified.

6 Why people would use it

People would use Pulse because it saves money and worry on a trip they already make every week, without the effort of finding partners each time. The target users are university students, faculty and staff, starting with AIUB, with any campus able to join.

Alternative	What it does well	What it lacks for campus commuting
Travelling alone (CNG, rickshaw, Uber, Pathao)	Available on demand	Full fare every trip; alone on late trips
Messenger groups and chance meetings	Free; partners are known classmates	No matching, money tracking or safety; breaks when schedules change
JyGo, BUET students, 2026 [4][5]	Zero-commission cost splitting for rickshaw, CNG, bike and car; saved routes; verification; live tracking; gender preference	City-wide and matched trip by trip; does not use class timetables or react to class changes
Carpus, Bilkent and METU students, Turkey [3]	Closed university networks; recurring weekly rides	Car-owner carpooling only; no hired vehicles, timetable forecasting or shared-fare settlement
University carpool portals: Purdue BoilerRide, CSU East Bay [6][7]	Match by route and class schedule	Static list of names; private cars only; no booking, money or safety features
TAGO; Liftango at UBC Okanagan [8][9]	Verified university community	Car-owner carpools built for North American campuses

Pulse answers the specific weaknesses of these tools:

Weakness seen today	Pulse feature
Last-minute cancellations and no-shows	A Bayesian reliability score used in matching, automatic standby replacement, and a one-tap "running late" signal

Weakness seen today	Pulse feature
Missed alerts and notification spam	A per-user notification budget, actionable alerts only, quiet hours, and delivery fallback from push to in-app to email
Distrust of strangers	Campus-email verification, matching that favours shared class sections, and a "why this crew" explanation
Door-to-door detours	Walkable meeting points chosen to minimise total detour
An empty app on day one	Route waitlists that unlock crews once enough people join; honest match odds and never fake users
Money disputes	Fares confirmed by members, receipts, a dispute flow, and a double-entry ledger in which money cannot appear or vanish
The same person always gets the worst deal	Long-term fairness that rotates booker duty, the longest detour and the last drop-off
Matching nobody can explain	An explanation on every match and an admin matching playground

Verification, live tracking, gender preference and zero commission already exist in JyGo, so this proposal does not claim them as new. All features are free.

7 Project objectives

1. **Read real timetable data.** Import the AIUB Fall 2026-2027 offered-course report and resolve each user's current classes from the grade report inside the browser. Verified by automated tests on the real files and at least 20 volunteer reports: every active class resolved, dropped classes excluded, and no grade data sent to the server.
2. **Form optimal, valid groups.** Plan each batch of a simulated AIUB day with 3,000 users, built from the real catalog, in under 5 seconds. Verified by property-based tests showing no group breaks capacity, time-window, detour or gender rules; by comparison with brute force on small cases, where 2-seat pairings must be optimal; and by a report of savings against travelling alone and against a greedy baseline.
3. **Make the AI measurably better than the simple version.** Forecast departure times as ranges (quantiles) and understand typed requests in English and Banglish. Verified on held-out data: the LightGBM forecast must beat the statistical baseline on quantile (pinball) loss, with its 80% ranges containing the actual time about 80% of the time; and the request model must beat the rule-based parser on field-level accuracy over a test set of sentences it never saw in training.
4. **Never lose or repeat an event.** Verified by fault-injection tests that cut network links and kill services mid-batch, with zero lost and zero double-applied events; by re-planning after a class change within 10 seconds; and by replaying a recorded day to identical results.
5. **Keep money exact and test with real people.** Verified by tests showing every ledger entry balances to zero, fair-split shares add up to the fare, and weekly settlement uses the proven minimum number of transfers; and by a survey of at least 50 AIUB students and a two-week pilot with at least 30 users.

8 Scope and boundaries

Included in this submission (every item is built and tested in full):

- Campus-email sign-in; roles for student, faculty, staff, shuttle operator, campus admin and platform admin; profiles and preferences

- Multi-campus data model protected by row-level security, with AIUB as the pilot campus
- Catalog import from the AIUB offered-course report; routine import from the grade report inside the browser; section search; staff office hours
- Campus onboarding agent that proposes a parser for a new university's timetable file, checked by a validator and approved by an admin
- Class change reports that re-plan affected trips
- Go now, plan for later and Autopilot with habit learning; typed requests in English and Banglish, understood by a small model trained in the AIUB AI Supercomputing Lab, with a rule-based parser as fallback
- Match-chance forecast (statistical baseline and LightGBM model) and the wait-or-go decision
- Matching for hired vehicles, own car or bike and bus companionship, with walkable meeting points, reliability scores, standby backfill and long-term fairness
- Road distances from OpenStreetMap, precomputed with Dijkstra's algorithm
- Shuttle runs with seat booking, optimal seat assignment and a simulated operator dashboard with a privacy-protected demand board
- Trip state machine; real-time chat; live location with GPS smoothing, map matching and route-deviation checks; SOS; safe-arrival checks with simulated SMS
- Assisted booking through Uber and Pathao
- Shapley fare split, double-entry ledger, optimal settlement with simulated bKash, disputes and receipts
- Notifications with per-user budgets and delivery fallback; calendar feed; savings card; route waitlists
- Ratings, reports, blocking and a safety centre
- Admin panel: catalog, calendar, zones and meeting points, operators and routes, reports, matching playground and analytics
- Event backbone (Apache Kafka, transactional outbox, schema registry), RabbitMQ job queues, campus simulator and shadow matching
- English interface; installable web app (PWA); complete GitHub pipeline with tests, CodeQL, Dependabot, image scanning, schema checks and the fault-injection suite

Outside the current scope: paid features (all features are free), real payments, real shuttle operators, real SMS delivery, native mobile apps, background location tracking, national ID or background checks, a large language model hosted in production, a Bangla interface, a fare estimate (Dhaka fares are negotiated and vary too much to predict reliably), a durable workflow engine (Temporal), Kubernetes, and a production monitoring stack.

Future improvements: a self-hosted language model for admin questions; a Bangla interface; Temporal for long-running workflows; production monitoring; real bKash and operator partnerships; a revenue model based on shuttle seats and university licences; native apps.

Assumptions and dependencies: the offered-course report and grade report keep their current formats; campus email domains are known; OpenStreetMap covers Dhaka's roads adequately; the Gemini free tier stays available and is used only for public timetable files; the AIUB AI Supercomputing Lab (NVIDIA DGX Spark systems), whose access the course faculty has confirmed, is used to train the request-understanding model; Uber's public deep links stay available [11], while Pathao, which publishes no deep-link format, opens with the destination copied; iPhone users add the app to the home screen to receive push notifications; development runs locally on an 8 GB laptop, and hosting for the pilot will be chosen before the pilot; enough volunteers join the pilot.

10 System features

Five user categories share one app, and each sees screens for its role. Every feature listed is essential and included in this submission.

User category: Student

A student wants a cheap, safe trip to or from campus, whether right after class or at any other time.

Page or screen	Actions available	Priority
Sign-in and onboarding	Verify campus email by code; set name, gender, home pin and trusted contact; install the app	Essential
Routine import	Upload the grade report (read in the browser) or search sections; confirm the week	Essential
Preferences	Allowed vehicles, gender preference, maximum wait, own vehicle, quiet hours for notifications	Essential
Home	Today's suggested trips with one-tap confirm; go now and plan buttons; type a request in English or Banglish	Essential
Go now	Choose direction; see likely partners and wait-or-go advice; wait for a match or go alone	Essential
Plan for later	Pick date, time, place and flexibility; repeat weekly	Essential
Autopilot	Turn on or off; see learned departure times; skip a day	Essential
Week view	Classes and planned trips for each day; report a class change	Essential
Route waitlist	See how many more people are needed on the route; invite classmates	Essential
Crew card	Members and why they match, vehicle, meeting point, booker and each member's share of the fare as a percentage; confirm, leave or mark running late; open Uber or Pathao with the trip filled in	Essential
Shuttle	Browse runs on the route; book or cancel a seat; boarding pass	Essential
Trip screen	Crew chat, live map, SOS, end trip, safe-arrival confirmation	Essential
Wallet	Enter fare and receipt; view ledger and fair-split breakdown; dispute an entry; settle weekly with simulated bKash	Essential
Safety centre	Rate co-riders, block, report; view own reliability score	Essential
Savings and calendar	Money saved this semester and a shareable card; add classes and trips to Google Calendar	Essential

User category: Faculty and staff

Faculty and staff use the same screens as students and travel in the same crews. Faculty pick the sections they teach, and staff enter fixed office hours, instead of uploading a grade report.

User category: Shuttle operator (simulated)

An operator runs vehicles on routes the system suggests from forecast demand.

Page or screen	Actions available	Priority
Fleet	Add vehicles, seats and drivers	Essential

Page or screen	Actions available	Priority
Demand board	See privacy-protected forecast riders by route and time; accept suggested runs	Essential
Runs	Publish runs with times, stops and seat price; see bookings	Essential
Run day	Mark departure and arrival; check in riders by boarding pass	Essential
Earnings	Simulated payouts per run	Essential

User category: Campus admin

A campus admin runs one university's setup each semester.

Page or screen	Actions available	Priority
Catalog	Upload the offered-course report; review import errors; activate the term	Essential
Onboarding agent	Upload a new timetable format; review the proposed parser and its validation report; approve or reject	Essential
Calendar	Add holidays, exam weeks and campus-wide class changes	Essential
Zones and meeting points	Draw home zones on a map; set gates and meeting points	Essential
Operators and routes	Approve operators and shuttle routes	Essential
Reports	Review safety reports and disputes; warn or suspend users	Essential
Matching playground	Replay a day step by step on a map; compare algorithms in shadow mode	Essential
Analytics	Users, trips, match rate, forecast accuracy, savings and fairness	Essential

User category: Platform admin

A platform admin adds campuses (name, email domains, location, catalog format) and appoints campus admins.

12 System architecture and backend modules

Pulse is a modular monolith with an event-driven backbone (Figure 2). A NestJS API handles user requests and stores state in PostgreSQL, publishes domain events to Kafka through a transactional outbox, background services react to those events, and RabbitMQ distributes one-off jobs.

- **Modular monolith.** The API is one application divided into modules with clear boundaries and one database. It is deliberately not a set of microservices, which would add network calls and deployment work without benefit for a team of three; a module can still be split out later.
- **Event-driven, not event-sourced.** PostgreSQL remains the source of truth. Events announce that something changed; they are not the data store.
- **Three ways to communicate.** Request and response between the user and the API; events (Kafka) between services; jobs (RabbitMQ) for work that must be done exactly once.
- **Functional core, imperative shell.** The matching, money and geo algorithms are pure functions with no database or network access, wrapped by NestJS, PostgreSQL and Kafka. This keeps them testable and replayable. The AI gateway and catalog parsers are interfaces with swappable implementations (ports and adapters).

The browser talks only to the Next.js server, which forwards API calls and the WebSocket to NestJS. When the API changes data, it writes the business rows and an outbox event in one database transaction, and a relay then publishes committed events to Kafka. Each service reads the log independently, does its work and writes results back the same way. Work that must be done exactly once, such as sending a notification, is placed on a RabbitMQ queue and handled by workers with retries. The API pushes updates to open browsers over WebSockets through Redis.

The code is one monorepo (pnpm workspaces and Turborepo). It holds apps for the web app, API and workers; a Python service for forecasting; a C++ tool for road distances; and shared packages for event and request schemas, the matching and money algorithms, catalog parsers and UI components. The algorithm packages are pure TypeScript with no database access, so they can be tested alone and replayed.

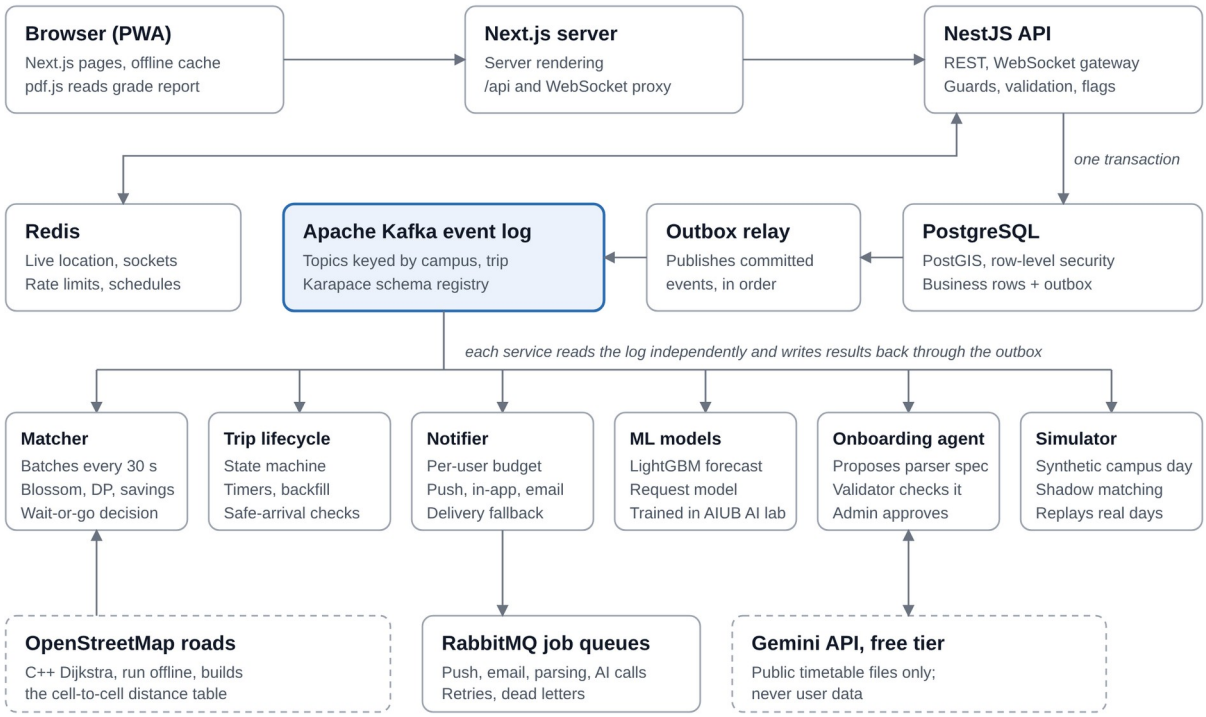


Figure 2. System architecture. Dashed boxes are outside data sources.

Module	Responsibility
Auth	Email-code sign-in, sessions, refresh tokens, rate limits
Campus	Campuses, email domains, terms, zones, gates and meeting points
Users	Profiles, preferences, vehicles, trusted contacts, reliability scores
Catalog	Timetable import through one parser per campus format; onboarding-agent specs
Routine	Enrollments from grade-report class IDs or search; staff office hours
Calendar	Holidays and class changes, published as events
Requests	Go now, plan for later and Autopilot; request-understanding model with rule-based fallback; habit learning
Forecast	Runs the forecast ONNX model; inputs for the wait-or-go decision
Matching	Batches, algorithms, fairness, meeting points
Crews and trips	Trip state machine, backfill, booker choice, assisted-booking links

Module	Responsibility
Shuttle	Operators, runs, optimal seat assignment, check-in, privacy-protected demand board
Realtime	WebSocket gateway, chat, live location, route-deviation checks
Notifications	Budgets, push, in-app and email with fallback, calendar feed
Ledger	Double-entry ledger, Shapley split, settlement, disputes, simulated bKash
Safety	Ratings, reports, blocks, SOS, safety checks
Events	Outbox relay, Kafka topics, schema registry, idempotent consumers, replay
Jobs	RabbitMQ exchanges and queues, retries with backoff, dead-letter handling
AI gateway	Routes AI tasks by data sensitivity; fallbacks, rate limits and feature flags
Admin	Catalog, moderation, matching playground, analytics

12.1 Event backbone and job queues

Apache Kafka 4, running as a single broker in KRaft mode [12], is the event log. It keeps every event in order for a retention period, and any number of services read it independently from their own position. That gives three things a job queue cannot: many independent readers, replay of history, and ordering per key. RabbitMQ [13] handles the other kind of message, jobs: tasks that exactly one worker must complete, such as sending a notification. Events record what happened; jobs say what to do about it.

One campus could run on PostgreSQL alone. Kafka is used because shadow matching, simulator replay and model retraining need replayable history read by several independent services; RabbitMQ is used because notifications, file parsing and AI calls need routing and per-job retries. The cost is one more service to run.

- **Transactional outbox.** Business rows and their event are written in one transaction, so the database and the log never disagree. The relay claims unsent rows with SELECT … FOR UPDATE SKIP LOCKED.
- **Idempotent consumers.** Delivery is at least once, so each consumer records processed event IDs in the same transaction as its effect, and duplicates change nothing.
- **Partition keys.** Requests are keyed by campus, so one matcher instance owns each campus without distributed locks. Trips are keyed by trip, ledger events by crew and class changes by section, so each sequence stays in order. Profiles live in a compacted topic that keeps the latest value per user.
- **Schemas.** Every event shares one envelope (ID, type, version, time, campus) and is registered in a schema registry (Karapace), with backward-compatibility checks in the pipeline.
- **Retries and dead letters.** Failing events move to retry topics, then to a dead-letter topic for inspection.
- **Job queues (RabbitMQ).** Notifications, file parsing, onboarding-agent calls, weekly settlement and model retraining are jobs on durable, replicated quorum queues, routed by exchanges. A worker acknowledges a job only after finishing it and takes a limited number at a time. Failed jobs wait in delay queues with fixed waiting times (time-to-live with dead-letter exchanges) for backoff, then move to a dead-letter queue.

12.2 Matching and optimisation algorithms

Every trip has one fixed end, the campus gate, which turns ride-sharing into a single-depot routing problem. Go now, plan for later and Autopilot all create the same request: direction, home area and

time window. Planned requests are matched ahead of time. Live requests are matched in batches every 30 seconds, so the engine sees all waiting people together.

Problem	Method	Complexity
Nearby candidates	H3 hexagonal grid with rings of neighbouring cells	$O(n \cdot k)$ instead of $O(n^2)$
Overlapping time windows	Sweep line with a min-heap of window ends	$O(n \log n)$
Pairs for 2-seat vehicles (rickshaw, bike)	Maximum-weight matching (Edmonds' blossom algorithm)	$O(V^3)$, exact
Groups of 3 or more (CNG, car)	Set partitioning by bitmask dynamic programming for small batches; Clarke–Wright savings with union-find, improved by large-neighbourhood search, for large ones	$O(3^n)$ for $n \le 15$; heuristic beyond
Shuttle seats	Min-cost max-flow	Polynomial, exact
Road distances	Dijkstra over the OpenStreetMap road graph from every H3 cell, precomputed offline in C++	O(1) lookup at run time
Meeting points	Point within a short walk that minimises total detour plus walking	Small search per group
Reliability	Beta–Bernoulli posterior of showing up	O(1) per update

Why the method changes with seat count. Pairing people for 2-seat vehicles is maximum-weight matching, which the blossom algorithm solves exactly in polynomial time. With 3 seats the problem becomes 3-dimensional matching, which is NP-hard, so Pulse solves small batches exactly and large ones with a heuristic measured against the exact answer. The heuristic merges groups in order of the distance saved by sharing, where G is the gate and d is road distance:

$$s(i, j) = d(G, i) + d(G, j) - d(i, j)$$

A merge is allowed only if seats remain, no member's detour exceeds 30%, time windows overlap, gender preferences hold and no member has blocked another. Scores add bonuses for shared class sections and long-term fairness, and subtract for low reliability. Ties are broken by fixed rules and all randomness is seeded, so replaying a day always gives the same result. The booker is the member dropped off last, who stays in the vehicle to pay the driver.

12.3 Forecasting and the wait-or-go decision

Habit learning. At first, a person is assumed to leave when their last class ends. Each real trip updates a blend of their own average delay and the campus average, weighted by their trip count n and a constant k. One unusual day barely moves it, and a steady habit takes over within weeks:

$$offset = (n \cdot \bar{x}_{user} + k \cdot \bar{x}_{campus}) / (n + k)$$

Forecast model. A LightGBM model, trained in Python, predicts departure-time quantiles from weekday, gaps between classes, exam weeks, rain and history. It is exported to ONNX and run inside NestJS, so no Python server is needed at run time. The statistical blend above is the baseline it must beat (Objective 3).

Wait or go. For a go-now request, the forecast gives the rate λ at which compatible people appear. The chance that at least one appears within t minutes is shown below. The app recommends waiting only while that chance times the expected saving is worth more than the user's time, and always offers going alone.

$$P(\text{partner within } t) = 1 - e^{-\lambda t}$$

12.4 AI integration and privacy techniques

AI tasks run through one gateway that labels each task by data sensitivity. Public data, such as a university's timetable file, may use the free Gemini tier. Personal data is never sent to it, because free-tier prompts may be used for training and read by reviewers [10]. Every AI task has a non-AI fallback, a rate limit and a feature flag.

- **Campus onboarding agent.** Given a new university's timetable file, the agent reads samples, proposes a declarative parser spec, and runs a deterministic validator over the whole file (valid days, start before end, unique class IDs per term). It revises the spec until validation passes, and an admin approves it before use. The model never runs code or sees user data.
- **Request understanding in English and Banglish.** A small multilingual language model is fine-tuned to pull the time, place, rider preference and maximum wait out of sentences such as "ML class er por Mirpur 10 jabo". Training runs in the AIUB AI Supercomputing Lab: a large open model hosted there helps write and label practice sentences, people check a sample, and the small model learns from them. The trained model is exported to ONNX and runs inside Pulse on an ordinary CPU, so typed requests never leave Pulse's servers and the app never depends on the lab while running. The rule-based parser remains as the fallback and as the baseline the model must beat.
- **Differential privacy.** Demand counts shown to shuttle operators receive calibrated Laplace noise, and counts below 3 are hidden, so no individual's travel can be inferred.

12.5 Money: fair split, ledger and settlement

Fair split. Each member pays their Shapley value: the average extra cost they add across every order in which the group could have formed. A rider whose home is on the way adds little and pays little. With at most four members, it is computed exactly from every subset of the group.

Double-entry ledger. Every money event is a journal entry whose postings sum to zero, stored append-only in whole paisa, so money cannot appear or vanish and every balance can be audited.

Optimal settlement. The minimum number of transfers equals the number of people minus the largest number of groups whose balances each sum to zero. Pulse finds it exactly with bitmask dynamic programming over subsets in $O(2^n \cdot n)$ time, which is fast for crew-sized groups.

12.6 Trip lifecycle and live safety

Each trip is a state machine stored in PostgreSQL (Figure 3). A scheduler claims trips whose next deadline has passed and applies the next transition, and every transition is published as an event, so timers survive restarts.

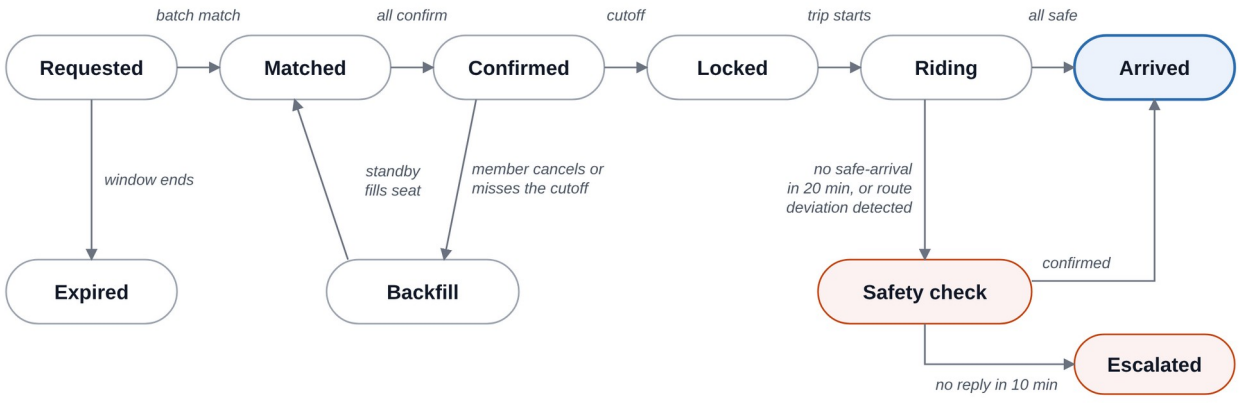


Figure 3. Trip lifecycle. Red states are safety states; an escalation alerts the trusted contact.

During a trip, phone GPS points are smoothed with a Kalman filter and snapped to roads with hidden-Markov-model map matching (the Viterbi algorithm). If the matched path leaves the expected route or stops for too long, the trip enters a safety check.

12.7 Simulator and shadow matching

A simulator builds a synthetic campus from the real catalog (homes, routines and habits) and publishes requests into the same event topics as real users. It drives load tests, tunes the wait-or-go rule and measures forecast accuracy. Recorded days can be replayed into a new matching version running as a separate consumer (shadow mode), comparing groups and savings without affecting users.

12.8 Quality engineering and local development

Every pull request runs a GitHub Actions pipeline: formatting, linting and type checks; unit and property-based tests for algorithms and money; integration tests against real PostgreSQL, Redis, Kafka and RabbitMQ containers; end-to-end browser tests with Playwright; the fault-injection suite (Toxiproxy and killed consumers); event-schema compatibility and database migration checks; CodeQL code scanning; Dependabot updates; secret scanning; and container image scans with a software bill of materials. Branches are protected, merges need a passing pipeline and a review, and major design choices are recorded as architecture decision records.

Development runs on one 8 GB laptop using Docker Compose profiles, so only the services in use are started. Kafka runs from its lightweight native image during development.

13 Preliminary database design

The database is PostgreSQL with PostGIS. Every table has a UUID primary key id unless a composite key is shown, money is stored in whole paisa, and row-level security limits every campus-scoped table to the user's campus. The catalog is stored once per term, and users point to sections instead of copying class times, so one class change reaches every affected user.

Entity or table	Key fields	Relationships
campus	id; name, slug, email_domains[], location, timezone	1 campus : N users, terms, zones, operators
term	id; campus_id (FK), name, starts_on, ends_on, is_active	1 term : N sections, calendar_events
section	id; term_id (FK), external_class_id (unique per term), course_code, title, section_label, type, enrolled, status	1 section : N meetings, enrollments
section_meeting	id; section_id (FK), weekday, start_time, end_time, room	N : 1 section
calendar_event	id; term_id (FK), section_id (FK, nullable), date, kind, new_start, new_end	N : 1 term; 0..1 section
user	id; campus_id (FK), email (unique), role, name, gender, phone, status	1 user : 1 preference, N enrollments, N trip_requests
preference	id; user_id (FK, unique), home_cell, zone_id (FK), gender_pref, modes[], max_wait_min, autopilot_on, quiet_hours	1 : 1 user
enrollment	id; user_id (FK), section_id (FK), role, source	N : M user and section
schedule_block	id; user_id (FK), weekday, start_time, end_time	N : 1 user (staff hours)

Entity or table	Key fields	Relationships
travel_habit	id; user_id (FK), weekday, direction, offset_min, trip_count	N : 1 user
reliability	id; user_id (FK, unique), shows, no_shows	1 : 1 user
fairness_balance	id; user_id (FK, unique), inconvenience_score	1 : 1 user
zone	id; campus_id (FK), name, area (polygon)	1 zone : N preferences, meeting_points
meeting_point	id; zone_id (FK), name, location (point), kind	N : 1 zone
cell_distance	(origin_cell, dest_cell) composite PK; metres, seconds	Lookup table, no FK
vehicle	id; owner_id (FK user), type, seats, plate	N : 1 user
trip_request	id; user_id (FK), kind, source (tap, text), direction, origin_cell, window_start, window_end, status, trip_id (FK, nullable)	N : 1 user; N : 0..1 trip
route_waitlist	id; zone_id (FK), user_id (FK), direction, joined_at	N : 1 zone; N : 1 user
crew	id; campus_id (FK), direction, mode, vehicle_id (FK, nullable), status	1 crew : N members, N trips
crew_member	id; crew_id (FK), user_id (FK), role, joined_at, left_at	N : M crew and user
trip	id; crew_id (FK), meeting_point_id (FK), planned_depart_at, status, next_action_at	1 trip : N participants, messages, journal entries
trip_participant	id; trip_id (FK), user_id (FK), confirmed_at, arrived_at, solo_distance_m	N : M trip and user
message	id; trip_id (FK), sender_id (FK user), body, created_at	N : 1 trip
ledger_account	id; user_id (FK), crew_id (FK), type	N : 1 user; 1 account : N postings
journal_entry	id; trip_id (FK, nullable), kind, receipt_key, created_at	1 entry : N postings (sum zero)
posting	id; entry_id (FK), account_id (FK), amount_paisa (signed)	N : 1 entry; N : 1 account
settlement	id; crew_id (FK), from_user (FK), to_user (FK), amount_paisa, status, reference	N : 1 crew
dispute	id; entry_id (FK), raised_by (FK user), reason, status	N : 1 journal_entry
operator	id; campus_id (FK), owner_id (FK user), name, status	1 operator : N vehicles, routes
shuttle_vehicle	id; operator_id (FK), type, seats, plate	N : 1 operator
shuttle_route	id; operator_id (FK), name, direction, stops (line), status	1 route : N runs

Entity or table	Key fields	Relationships
shuttle_run	id; route_id (FK), vehicle_id (FK), departs_at, seat_price_paisa, status	1 run : N seat_bookings
seat_booking	id; run_id (FK), user_id (FK), status, checked_in_at	N : M run and user
rating, report, block	id; author_id (FK), subject_id (FK user), trip_id (FK, nullable), score or reason, status	N : 1 user on both sides
sos_event	id; trip_id (FK), user_id (FK), location, created_at, resolved_at	N : 1 trip
trusted_contact	id; user_id (FK), name, phone	N : 1 user
notification	id; user_id (FK), kind, channel, status, sent_at, delivered_at	N : 1 user
push_subscription	id; user_id (FK), endpoint, keys	N : 1 user
onboarding_spec	id; campus_id (FK), spec, validation_report, status, approved_by (FK user)	N : 1 campus
ml_model	id; kind (forecast, request_parser), version, trained_at, metrics, artifact_key, is_active	No FK
outbox_event	id; aggregate, aggregate_id, type, payload, created_at, published_at	Written with each business change
processed_event	(consumer, event_id) composite PK; processed_at	One row per consumer per event
audit_log	id; actor_id (FK user), action, entity, entity_id, at	N : 1 user

Live locations are not stored in this database. They stay in Redis only during a trip and expire when it ends.

14 Authentication, security and privacy

A timetable plus a home location reveals when and where a person travels every week, so privacy is a core design rule. No other user ever sees someone's routine, exact home or reliability score.

Sign-in. Users sign in with a 6-digit code sent to a campus email address, and only domains registered for that campus are accepted. Codes are stored hashed, expire after 10 minutes and lock after 5 wrong attempts. Requests are rate-limited per email and IP address, and sign-up is protected by a bot check. There are no passwords to leak, and the app never asks for university portal passwords.

Sessions. A short-lived access token is paired with a refresh token in an httpOnly, Secure, SameSite cookie on the app host only. Refresh tokens are stored hashed and rotated on each use; reuse of an old token signs out every session of that user.

Roles, ownership and tenancy. NestJS guards check the role on every route. Row-level security in PostgreSQL limits every query to the user's campus, so even a faulty query cannot return another campus's data. A user can read a crew, trip, chat or ledger only if they belong to it.

Grade report. The grade report contains grades and CGPA, which Pulse must never hold. It is parsed inside the browser with pdf.js: only the term name and the current semester's class IDs are sent, and the file never leaves the device. The server checks each class ID against that term's catalog and the course title, because AIUB reuses class IDs in different semesters.

Input and uploads. All requests are validated against shared schemas, and unknown fields are rejected. The database is accessed only through parameterised queries. Uploads (timetable files, receipts) are checked for type and size, stored privately, read through short-lived links and parsed by a worker, not the API.

AI and outside services. Only public timetable files are sent to the free Gemini tier. The agent outputs a declarative spec that a validator checks, so model output never runs as code. The request-understanding model runs inside Pulse, so typed requests are never sent to an outside AI service. Sentences from real users are used for training only with their consent and after removing personal details. Every AI feature can be switched off with a feature flag and has a non-AI fallback.

Personal data.

- Home is stored as a coarse map cell, never a street address.
- Co-riders see only first name, photo, department and home zone, and only after a crew forms. Phone numbers stay hidden behind in-app chat.
- Live location is shared only during a trip, only with that crew, and is deleted when the trip ends.
- Gender is self-declared and used only for gender-preference matching.
- Demand shown to operators is protected with differential privacy, and counts below 3 are hidden.
- Users can download their data and delete their account. Logs leave out personal fields, and admin actions are written to an audit log.

Software supply chain. CodeQL, Dependabot, secret scanning and container image scans run in the GitHub pipeline. Hosting security will be specified when the hosting setup is chosen; the design already assumes HTTPS everywhere and no database exposed to the internet.

References

[1] AIUB Offered Course Report, Fall 2026-2027, exported from the AIUB student portal.

[2] Kimola, BlaBlaCar user feedback analysis (Google Play). https://kimola.com/reports/insightful-blablacar-user-feedback-analysis-report-google-play-156428

[3] Carpus: campus carpooling pitch, Bilkent University, 2026. https://fba.bilkent.edu.tr/wp-content/uploads/2026/06/Carpus.pdf

[4] JyGo on Product Hunt. https://www.producthunt.com/p/jygo

[5] BUET URP: students launch vehicle-pooling app JyGo, 18 April 2026. https://urp.buet.ac.bd/Web/NewsAndEvents/60068

[6] Purdue University, BoilerRide carpool matching, 2009. https://www.purdue.edu/uns/insidepurdue/2009/090304_boilerride.html

[7] CSU East Bay, carpool to school. https://www.csueastbay.edu/parking/alt-trans/rideshare-carpool/carpool-to-school.html

[8] TAGO Rides, App Store. https://apps.apple.com/app/id6763382426

[9] UBC Okanagan partners with carpool app Liftango. https://parking.ok.ubc.ca/?p=1731

[10] Gemini API pricing changes, 2026 (agentdeals.dev). https://agentdeals.dev/gemini-api-pricing-changes

[11] Uber Developers, Introduction to Deep Links. https://developer.uber.com/docs/deep-linking

[12] Apache Kafka 4.0.0 release announcement, 18 March 2025. https://kafka.apache.org/blog/2025/03/18/apache-kafka-4.0.0-release-announcement/

[13] RabbitMQ 4.0: what is new. https://blog.rabbitmq.com/docs/4.0/whats-new